

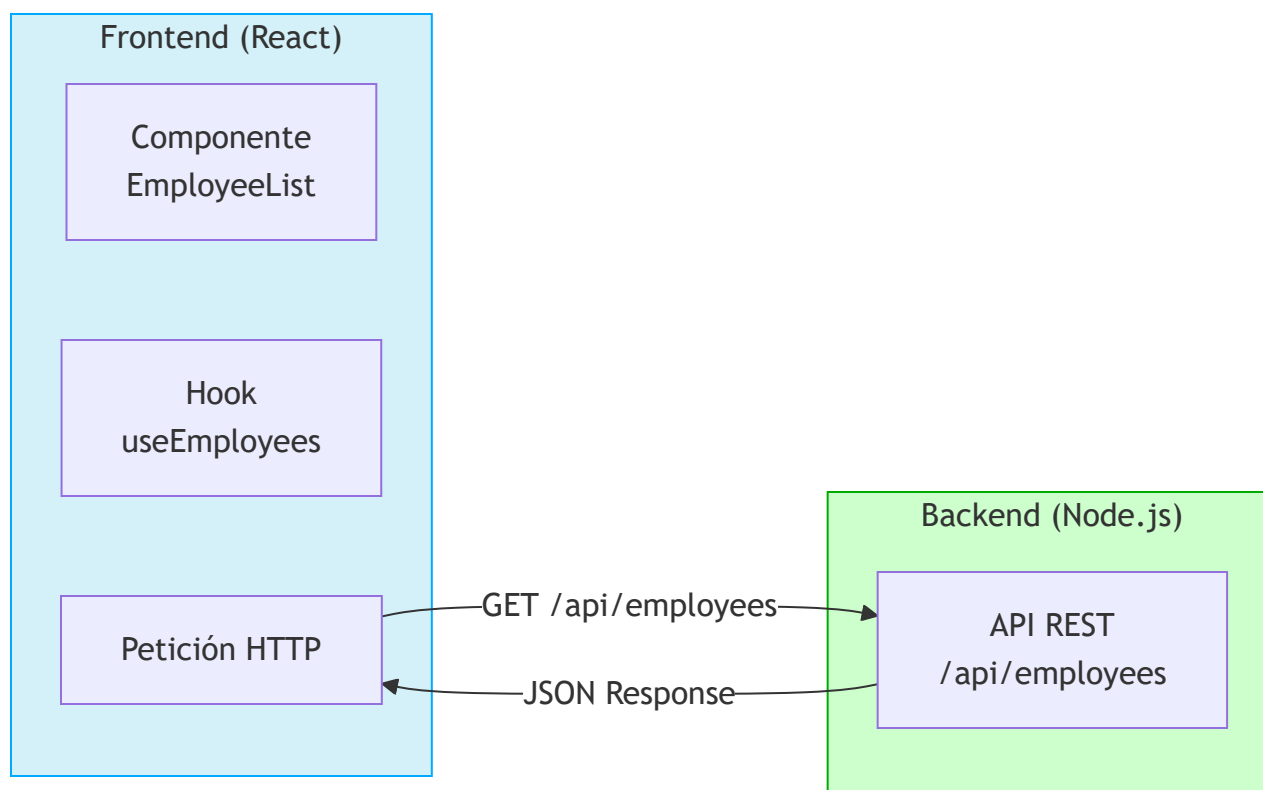
Guía Didáctica: Fetch vs Axios - Comunicación con el Backend

Objetivo de la Guía

Vamos a aprender:

1. Qué es FETCH (nativo de JavaScript)
2. Qué es AXIOS (librería externa)
3. Cuáles son las DIFERENCIAS entre ellos
4. Por qué usar AXIOS en proyectos profesionales
5. Cómo IMPLEMENTAR cada uno en tu proyecto

El Problema: Comunicación con el Backend



El reto: Necesitamos una forma de hacer peticiones HTTP desde React hacia el backend.



Opción 1: Fetch (Nativo de JavaScript)

¿Qué es Fetch?

FETCH es la API NATIVA de JavaScript para hacer peticiones HTTP.
Está integrada en todos los navegadores modernos.
NO necesita instalación.

Sintaxis Básica

// 📦 Ejemplo básico con fetch

// GET - Obtener datos

```
fetch('http://localhost:3000/api/employees')  
  .then(response => response.json()) // ← Convertir a JSON  
  .then(data => console.log(data))   // ← Usar los datos  
  .catch(error => console.error(error));
```

// POST - Enviar datos

```
fetch('http://localhost:3000/api/employees', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json',  
  },  
  body: JSON.stringify({  
    cedula: '1234567890',  
    nombres: 'Ana García',  
    salario: 1500  
  })  
})  
  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error(error));
```

Fetch en tu Proyecto

```
// 📁 src/api/employee.api.ts (con fetch)
```

```
const API_URL = 'http://localhost:3000/api/employees';
```

```
export const EmployeeAPI = {
```

```
  // ✅ GET - Obtener todos los empleados
```

```
  async getAll(): Promise<Employee[]> {
```

```
    const response = await fetch(API_URL);
```

```
    if (!response.ok) {
```

```
      throw new Error(`Error ${response.status}: ${response.statusText}`);
```

```
    }
```

```
    const data = await response.json();
```

```
    return data.data;
```

```
  },
```

```
  // ✅ POST - Crear empleado
```

```
  async create(data: CreateEmployeeRequest): Promise<Employee> {
```

```
    const response = await fetch(API_URL, {
```

```
      method: 'POST',
```

```
      headers: {
```

```
        'Content-Type': 'application/json',
```

```
      },
```

```
      body: JSON.stringify(data),
```

```
    });
```

```
    if (!response.ok) {
```

```
      const error = await response.json();
```

```
      throw new Error(error.message || 'Error al crear empleado');
```

```
    }
```

```
    const result = await response.json();
```

```
    return result.data;
```

```
  },
```

```
  // ✅ PUT - Actualizar empleado
```

```
  async update(id: string, data: UpdateEmployeeRequest): Promise<Employee> {
```

```
    const response = await fetch(`${API_URL}/${id}`, {
```

```
      method: 'PUT',
```

```
      headers: {
```

```

        'Content-Type': 'application/json',
    },
    body: JSON.stringify(data),
  });

  if (!response.ok) {
    throw new Error(`Error ${response.status}: ${response.statusText}`);
  }

  const result = await response.json();
  return result.data;
},

// ✅ DELETE - Eliminar empleado
async delete(id: string): Promise<void> {
  const response = await fetch(`${API_URL}/${id}`, {
    method: 'DELETE',
  });

  if (!response.ok) {
    throw new Error(`Error ${response.status}: ${response.statusText}`);
  }
},

};

```

Ventajas de Fetch

- ✅ NO requiere instalación (es nativo)
- ✅ Está disponible en todos los navegadores modernos
- ✅ Promesas nativas (async/await)
- ✅ Ligero (sin dependencias externas)

Desventajas de Fetch

- ❌ Verboso (mucho código para cosas simples)
- ❌ No maneja errores automáticamente (hay que verificar response.ok)
- ❌ No tiene interceptores
- ❌ No tiene timeout nativo
- ❌ No cancela peticiones automáticamente
- ❌ CORS puede ser problemático



Opción 2: Axios (Librería Externa)

¿Qué es Axios?

AXIOS es una librería externa para hacer peticiones HTTP.

Se instala con npm.

Ofrece una API más SIMPLE y PODEROSA que fetch.

Instalación

```
npm install axios
```

Sintaxis Básica

```
// 📁 Ejemplo básico con axios
```

```
import axios from 'axios';
```

```
// GET - Obtener datos
```

```
axios.get('http://localhost:3000/api/employees')  
  .then(response => console.log(response.data)) // ← data ya está parseada  
  .catch(error => console.error(error));
```

```
// POST - Enviar datos
```

```
axios.post('http://localhost:3000/api/employees', {  
  cedula: '1234567890',  
  nombres: 'Ana García',  
  salario: 1500  
})  
  .then(response => console.log(response.data))  
  .catch(error => console.error(error));
```

Axios en tu Proyecto

1. Cliente Configurado

```
// 📁 src/api/client.ts

import axios from 'axios';

const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:3000';

export const apiClient = axios.create({
  baseURL: API_BASE_URL,      // ← URL base
  headers: {
    'Content-Type': 'application/json',
  },
  timeout: 10000,             // ← ¡TIME OUT NATIVO!
});

// ✅ Interceptor de REQUEST (log de peticiones)
apiClient.interceptors.request.use((config) => {
  console.log(`🔗 ${config.method?.toUpperCase()} ${config.url}`);
  return config;
});

// ✅ Interceptor de RESPONSE (log de respuestas + manejo de errores)
apiClient.interceptors.response.use(
  (response) => {
    console.log(`✅ ${response.status} ${response.config.url}`);
    return response;
  },
  (error) => {
    console.error(`❌ Error: ${error.message}`);
    return Promise.reject(error);
  }
);
```

2. API con Axios

```
// 📁 src/api/employee.api.ts (con axios)
```

```
import { apiClient } from './client';
```

```
import type { Employee, CreateEmployeeRequest, UpdateEmployeeRequest } from '../types/employee.1
```

```
const API_URL = '/api/employees';
```

```
export const EmployeeAPI = {
```

```
  // ✅ GET - Obtener todos
```

```
  async getAll(): Promise<Employee[]> {
```

```
    const response = await apiClient.get<{ data: Employee[] }>(API_URL);
```

```
    return response.data.data;
```

```
  },
```

```
  // ✅ POST - Crear
```

```
  async create(data: CreateEmployeeRequest): Promise<Employee> {
```

```
    const response = await apiClient.post<{ data: Employee }>(API_URL, data);
```

```
    return response.data.data;
```

```
  },
```

```
  // ✅ PUT - Actualizar
```

```
  async update(id: string, data: UpdateEmployeeRequest): Promise<Employee> {
```

```
    const response = await apiClient.put<{ data: Employee }>(
```

```
      `${API_URL}/${id}`,
```

```
      data
```

```
    );
```

```
    return response.data.data;
```

```
  },
```

```
  // ✅ DELETE - Eliminar
```

```
  async delete(id: string): Promise<void> {
```

```
    await apiClient.delete(`${API_URL}/${id}`);
```

```
  },
```

```
  // ✅ PATCH - Aumentar salario
```

```
  async increaseSalary(id: string, percentage: number): Promise<Employee> {
```

```
    const response = await apiClient.patch<{ data: Employee }>(
```

```
      `${API_URL}/${id}/salary`,
```

```
      { percentage }
```

```
    );
```

```
    return response.data.data;
```

```
    },  
  };
```

Ventajas de Axios

- ✓ Código más LIMPIO y LEGIBLE
- ✓ Manejo de errores AUTOMÁTICO
- ✓ INTERCEPTORES (request/response)
- ✓ TIME OUT nativo
- ✓ Cancelación de peticiones
- ✓ Transformación de datos automática
- ✓ Soporte para progreso de descarga/subida
- ✓ Funciona en Node.js y navegador

Desventajas de Axios

- ✗ REQUIERE instalación (dependencia externa)
- ✗ Aumenta el tamaño del bundle
- ✗ Curva de aprendizaje (aunque es pequeña)



Comparativa Detallada: Fetch vs Axios

● AXIOS (Librería)

✗ Requiere instalación

✓ Código limpio

✓ Interceptores

✓ Timeout nativo

✓ Cancelación automática

✓ JSON parseado automático

● FETCH (Nativo)

✓ Nativo - sin instalación

✗ Verboso

✗ No tiene interceptores

✗ No tiene timeout nativo

✗ No cancela peticiones

✗ Hay que parsear JSON manualmente

Tabla Comparativa

Característica	Fetch	Axios
Instalación	✓ Nativo	✗ <code>npm install axios</code>
Sintaxis	✗ Verbosa	✓ Limpia
JSON parsing	✗ Manual (<code>response.json()</code>)	✓ Automático (<code>response.data</code>)

Característica	Fetch	Axios
Errores HTTP	✗ No lanza error en 4xx/5xx	✓ Lanza error automáticamente
Interceptores	✗ No tiene	✓ Sí tiene
Timeout	✗ No tiene nativo	✓ Sí tiene (<code>timeout: 10000</code>)
Cancelación	✗ Compleja (AbortController)	✓ Simple (<code>CancelToken</code>)
Progreso	✗ No tiene	✓ Sí tiene (<code>onUploadProgress</code>)
Transformación	✗ Manual	✓ Automática (<code>transformRequest</code>)
Base URL	✗ Manual	✓ Sí (<code>baseUrl</code>)
Headers globales	✗ Manual	✓ Sí (en <code>defaults</code>)
Bundle size	✓ 0 KB	✗ ~8 KB (gzip)



Ejemplo Comparativo: GET /api/employees

Con Fetch

// 📁 Con Fetch

```
async function getEmployees() {
  try {
    const response = await fetch('http://localhost:3000/api/employees');

    // ❌ Hay que verificar el error manualmente
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    // ❌ Hay que parsear JSON manualmente
    const data = await response.json();
    return data.data;
  } catch (error) {
    console.error('Error:', error);
    throw error;
  }
}
```

Con Axios

// 📁 Con Axios

```
async function getEmployees() {
  try {
    // ✅ Error lanzado automáticamente en 4xx/5xx
    // ✅ JSON parseado automáticamente
    const response = await apiClient.get('/api/employees');
    return response.data.data; // ← Ya está parseado
  } catch (error) {
    console.error('Error:', error);
    throw error;
  }
}
```

🔥 Lo que Axios Hace por Ti (Automáticamente)

Axios lo hace automático

✓ Parsear JSON

✓ Manejar errores HTTP

✓ Añadir headers

✓ Timeout

✓ Logging con interceptores

✓ Base URL

Fetch vs Axios: Código Necesario

Acción	Fetch	Axios
Configurar base URL	fetch(url) (siempre completa)	baseUrl en axios.create()
Headers por defecto	Cada petición	headers en axios.create()
Parsear JSON	response.json()	✓ Automático
Manejar error 404	if (!response.ok)	✓ Automático (catch)
Timeout	Manual con AbortController	timeout: 10000
Logging	Manual	Interceptores

🎯 ¿Cuál Usar en tu Proyecto?

Usa FETCH si...

- ✓ Es un proyecto pequeño o prototipo
- ✓ No quieres dependencias externas
- ✓ Estás aprendiendo y quieres entender lo nativo
- ✓ Solo necesitas hacer GET/POST simples

Usa AXIOS si...

- ✅ Es un proyecto profesional/empresarial
- ✅ Necesitas interceptores (autenticación, logging)
- ✅ Necesitas timeout para peticiones lentas
- ✅ Quieres código más limpio y mantenible
- ✅ Necesitas cancelar peticiones
- ✅ Quieres manejar errores de forma consistente

Recomendación para tu Proyecto

📌 RECOMENDACIÓN: USA AXIOS

Tu proyecto ya usa axios (lo vimos en los archivos).
Es la opción preferida en la industria para proyectos profesionales.

BENEFICIOS EN TU PROYECTO:

- 💠 Interceptores: logging automático de peticiones
- 💠 Timeout: 10 segundos de espera
- 💠 Base URL: configurada en .env
- 💠 Manejo de errores: centralizado
- 💠 Código limpio: EmployeeAPI con axios



Resumen Final

FETCH VS AXIOS

● FETCH:

- ✓ Nativo, sin instalación
- ✗ Verboso, mucho código boilerplate
- ✗ No tiene interceptores
- ✗ No tiene timeout nativo

● AXIOS:

- ✗ Requiere instalación
- ✓ Código limpio y legible
- ✓ Interceptores
- ✓ Timeout nativo
- ✓ Cancelación de peticiones
- ✓ JSON parseado automático

🎯 RECOMENDACIÓN:

- Usa AXIOS en proyectos profesionales
- Usa FETCH para prototipos o aprendizaje

Preguntas de Reflexión

1. ¿Por qué axios es más popular que fetch en proyectos grandes?
 - Porque ofrece interceptores, timeout, mejor manejo de errores.
2. ¿Qué son los interceptores y por qué son útiles?
 - Son funciones que se ejecutan antes/después de cada petición.
 - Útiles para: autenticación, logging, manejo global de errores.
3. ¿Por qué el timeout es importante?
 - Para evitar que la aplicación se quede "colgada" esperando una respuesta.
4. ¿Qué pasaría si usas fetch en un proyecto grande?
 - Tendrías que escribir mucho código repetitivo para:
 - Manejar errores
 - Parsear JSON
 - Configurar headers
 - Timeout